

GMXTRANSPLANT

MD System Builder

Version 1.0.1

Complete documentation for the command-line interface and the desktop application, assembled from the guides shipped with this release.

Generated 2026-09-24.

Contents

Overview	3
Choose a task	3
Install	3
Desktop application	4
Quick start	5
Prepare your inputs	5
Optional OpenMM minimization	7
Assembly checks and outputs	8
Limitations	9
Development	9
Runnable protein insertion example	9
Desktop application	11
Install and launch	11
Your first run	11
Documentation	12
macOS and MacBook Air	13
Output layout and replacement	13
Examples	14
Cancellation and testing	14
Two-folder transplant and comparison views	14
CHARMM-GUI transplant mode	15
Detection and selection	15
Optional YAML	16
Alignment, topology and outputs	16
GUI, examples and visualization	17
Comparison views	18
Portable and offline use	19

Overview

New first mode: two-folder CHARMM-GUI protein transplantation. Supply only reference and transplant folders; proteins and ligands are detected and replaced automatically. The other assembly modes remain available. See the mode guide (CHARMPROT.md), supplied example (examples/charmprot/README.md), and PyMOL/VMD comparisons (VISUALIZATION.md).

GMXtransplant assembles membrane-protein systems using an existing prepared membrane, solvent, and ion environment. You can replace a protein or protein-ligand complex, replace a bound ligand, or restore experimentally resolved cholesterol.

Two ways to work, one pipeline. The desktop application is the main way to use GMXtransplant: browse for inputs, adjust settings in a guided form, run the bundled examples in one click, and follow progress and results in one window. For scripts, clusters and remote servers, every mode is also available as a complete command-line tool (gmxttransplant). Both run the same pipeline and use the same YAML configurations: every application run saves its setup as run.yaml in the output folder, which runs unchanged with gmxttransplant --mode -i run.yaml.

This is a structure-assembly tool and expects every molecular component to be parameterized already. After assembly, inspect the outputs and use them directly, or prepare portable OpenMM inputs for restrained clash relaxation.

Choose a task

Task	Command option	Example configuration
Replace proteins and ligands using only two CHARMM-GUI folders	--mode charmprot	charmprot.yaml
Insert a protein or protein-ligand complex into a prepared environment	--mode protein	protein_replace.yaml
Replace a bound ligand while retaining the protein	--mode lig	ligand_replace.yaml
Restore experimental cholesterol and adjust leaflet composition	--mode chl	cholesterol_restore.yaml

Protein mode supports membrane proteins generally, including receptors, channels, and transporters. Any ligands included in the incoming selection move with the protein during alignment.

Install

Use Python 3.10 or newer on Linux or macOS. From the downloaded or cloned source directory, one command installs the desktop application, the command-line tool and everything they need:

```
python3 -m pip install .
gmxttransplant-gui      # desktop application
gmxttransplant --help   # command-line tool
```

A separate environment keeps GMXtransplant's dependencies apart from other software: python3 -m venv .venv && source .venv/bin/activate before installing (or conda create -n gmxttransplant python=3.12 && conda activate gmxttransplant).

What is installed, and why

Package	Why GMXtransplant needs it
NumPy, SciPy	Coordinate arithmetic, rigid-body fitting and distance searches
PYYAML	Reading and writing the YAML configurations
MDAnalysis	Reading and writing GRO/PDB/MOL2 structures and periodic-box geometry
PySide6 (Qt)	The desktop application's windows. The command-line tool never loads it, so it also works on servers without a display
openbabel-wheel (Open Babel)	The obabel command that adds hydrogens and CHARMM36 names to experimental cholesterol (cholesterol restoration with convert_to_charmm36: t

Optional, installed separately

Package	Needed for	How to install
OpenMM	Running a prepared minimization folder	<code>python3 -m pip install "[minimize]"</code> or <code>conda install</code>
PyMOL (with pymol2)	The <code>pymol_align</code> / <code>pymol_cealign</code> alignment methods; the default <code>mask_fit</code> does not need it	<code>conda install -c conda-forge pymol-open-source</code>
VMD	The Open in VMD comparison view	From the VMD website

On macOS, PyMOL and VMD installed as applications in /Applications are found automatically. For a copy elsewhere, set `GMXTRANSPLANT_PYMOL` or `GMXTRANSPLANT_VMD` to its full path.

If a dependency cannot be installed

- Open Babel on macOS 12 or 13: no ready-built pip package exists, so it is

skipped. Install it with `brew install open-babel` or `conda install -c conda-forge openbabel`. On other systems where the pip package fails, install GMXtransplant with `GMXTRANSPLANT_NO_OPENBABEL=1 python3 -m pip install .` and use one of those commands (or `sudo apt-get install openbabel`). Only cholesterol restoration needs it; check with `obabel -V`.

- Qt on older Linux systems or some clusters: install just the command-line

tool with `GMXTRANSPLANT_CLI_ONLY=1 python3 -m pip install ..` Every mode, example and option remains available through `gmxtransplant`; the application can be added later with `python3 -m pip install "PySide6>=6.6,<7"`.

For an editable installation, use `python3 -m pip install -e ..` To run the command line from source without installing the package, use `python3 run_pipeline.py` in place of `gmxtransplant`.

Desktop application

Launch the application from any directory:

```
gmxtransplant-gui
# Equivalent module entry point:
python -m gmxtransplant.gui
```

The desktop application supports macOS, Linux, and WSL 2 with WSLg. It includes an input editor for all configuration sections, file and folder browsing, configuration import/export, one-click examples, validation, readable progress summaries, cancellation, and result browsing. Detailed logs are collapsed until View Command Progress is clicked. YAML is generated automatically in the background; there is no YAML editing tab. Select an output folder first. Runs write directly to `charmprot/`, `protein/`, `ligand/`, or `cholesterol/`; example runs use `examples/`. Repeating a run replaces its generated products, including `toppar/`, without timestamped run directories.

Complete example inputs are included in the package for offline use. This adds approximately 58 MB of installed data (the full wheel is about 12 MB compressed). Read the GUI guide (`GUI_USAGE.md`) for path handling, output replacement, and WSL setup.

On a MacBook Air, run these commands in Bash (or zsh) from the repository root:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install .
gmxtransplant-gui
```

Use a native Python installation matching your Mac's architecture. See the macOS instructions (`GUI_USAGE.md#macos-and-macbook-air`) for prerequisites (including Open Babel on macOS 12 and 13).

Quick start

In the desktop application

1. Run gmxtransplant-gui and choose an output folder at the top of the window.
2. Open the Examples tab and click Run example on any card to see a complete run, or choose a mode on the Configuration tab and browse for your own inputs.
3. Click Check configuration, then Run pipeline, and follow the run on Run & results.

On the command line

Work in a separate directory and generate the example for your task:

```
mkdir my_gmxtransplant_job
cd my_gmxtransplant_job
gmxtransplant --show-example protein > protein_replace.yaml
```

Edit the paths, residue selections, topology sources, and scientific settings. Then validate the configuration and run assembly:

```
gmxtransplant --mode protein -i protein_replace.yaml --dry-run
gmxtransplant --mode protein -i protein_replace.yaml --output results/
```

--dry-run checks the configuration without opening molecular input files. Input identity, topology, and geometry checks happen during the normal run.

--output DIR chooses where the generated files go. Without it they are written to the directory you ran the command from. Either way, the run's closing line names the absolute folder it wrote to.

Use --show-example lig or --show-example chl for the other modes. Add --no-comments to print a compact example with the same active settings.

Source-checkout example configurations are available for protein insertion (examples/protein_insertion/README.md), ligand replacement (examples/ligand_replacement/README.md), and cholesterol restoration (examples/cholesterol_restoration/README.md). The ligand and cholesterol examples contain input folders for your own structures. Their READMEs list the exact filenames and settings to supply.

Prepare your inputs

The target environment can come from CHARMM-GUI, another system builder, or an MD trajectory frame. Supply matching coordinate and GROMACS topology files, including all force-field and molecule definitions. Incoming proteins and ligands also need matching parameters from a compatible force field.

GRO is preferred: it preserves the periodic box and four-character residue names such as POPC and TIP3. If using an MD frame, preserve atom order, whole molecules, and the correct box. Validated periodic processing requires an axis-aligned orthorhombic cell.

Example files group paths under paths: and reuse them as "\${name}". Unknown or duplicate YAML keys are rejected.

Every input path must be written out in full. No input is resolved against the working directory, against the configuration file's folder, or against any shared input root, and a relative input path is rejected with a message naming the offending key. The one exception is the examples shipped with the package: their paths resolve against the examples/ folder installed alongside the code, so they run anywhere without referring to your filesystem.

Generated files are the mirror image: an output name such as step5_input.gro is a plain filename that lands in the run's output folder. That folder is --output DIR when you pass it, and otherwise the directory you ran the

command from. Every run ends by printing the absolute path it wrote to.

Protein insertion

Set these together in `protein_replace.yaml`:

- `target_box.protein_mask`: the complete old protein/ligand block to remove.
- `replacement_structure.protein_mask`: the complete incoming block to insert, including any bound ligands.
- `alignment`: corresponding protein atoms used for fitting.
- `replacement_ligands`: coordinate residue names and ITP files for incoming ligands; charge and molecule type are read automatically from the ITP. Use `[]` when there are none. Each ligand may also give `forcefield_path` (see below).
- `topology.protein_toppar_dir` and `topology.environment_toppar_dir`: the protein and retained environment parameter sources.

The alignment transform is applied to the whole incoming block. Fit a stable protein region so that a changed ligand pose does not bias placement. For example, a receptor complex can be aligned using corresponding receptor backbone atoms.

`mask_fit` requires matching atom-name order and residue grouping, equal atom counts, and at least three non-collinear points. Use PyMOL alignment when correspondence must be established between different protein sequences.

Ligand replacement

In `ligand_replace.yaml`, select the prepared system and the original ligand residue. Use `RESNAME:N` to select a specific occurrence when its name appears more than once.

The incoming ligand needs three files: its coordinates (`new_ligand.coord_path`, e.g. a `.mol2`), its ITP (`new_ligand.itp_path`), and the force field holding its bond, angle and dihedral parameters (`new_ligand.forcefield_path`). A CHARMM-GUI ligand ITP only lists which atoms are bonded; the parameters are in the `forcefield.itp` CHARMM-GUI generated together with it. When `forcefield_path` is empty, a `forcefield.itp` next to the ITP is used, otherwise the environment's. Before writing the topology the run checks that every bonded term of the new ligand has parameters, and stops naming any that are missing.

Placement	Behavior
<code>autofit</code>	Fit matching, uniquely named heavy atoms
<code>pairfit</code>	Fit explicit pairs of atom names
<code>mcsfit</code>	Detect the largest shared substructure automatically (same elements and bonds), fit on its closest symmetric match, and report the pairs
<code>nofit</code>	Keep incoming coordinates that are already in the target frame

The protein is retained and protected from clash-based removal. Protein-ligand contacts are reported for inspection.

Cholesterol restoration

In `cholesterol_restore.yaml`, the experimental protein is aligned to the target protein and the associated cholesterol coordinates are transferred.

Optional CHARMM36 conversion regenerates hydrogens with Open Babel and validates the result against the included cholesterol reference. The example explains the limits for reconstructing missing heavy atoms.

Leaflet composition settings can remove surplus molecules outside the protected region around the protein. They do not create missing lipids or solvent.

This restoration option supports cholesterol only. There is currently no corresponding option to preserve and transfer other experimentally resolved bound lipids. Existing lipids in the target membrane are retained according to the clash and composition settings.

Residue selections

Masks use one-based sequential residue positions in coordinate-file order, not the residue numbers printed in PDB or GRO files.

Mask	Selection
:1-298	All atoms in residues 1 through 298
:1,5,10-20	Selected residues or ranges
:1-298@CA	CA atoms in those residues
:1-298@C,CA,N,O	Backbone atoms in those residues
:1-298&!:297-298	Exclude residues 297 and 298

Optional OpenMM minimization

Prepare one openmm_minimization/ folder for a short clash-relaxation step. Preparation does not run or submit minimization. Execute the generated runner separately when ready.

Harmonic positional restraints keep protein and ligand heavy atoms and lipid head groups (the polar P, O, N and S atoms of lipids and sterols) near their starting coordinates. Lipid tails, hydrogens, water, and ions relax freely. These restraints allow small movements; they do not freeze the protein, ligand or membrane.

Prepare the folder

Use case	Command
Assemble and then prepare inputs from that run	gmxtransplant --mode protein -i protein_replace.yaml --prepare-minimization
Prepare inputs from an existing matching system	gmxtransplant prepare-minimization --coordinates step5_input.gro --topology topol.top

--prepare-minimization is an assembly option and works with all four assembly modes. It requires topology.enabled: true and uses the freshly generated GRO and topology. Setting minimization.enabled: true in the assembly YAML has the same effect.

prepare-minimization is a separate command for existing files and does not repeat assembly. Use --include-dir for an additional topology search directory and --output to choose a different destination.

For adjustable standalone settings:

```
gmxtransplant --show-example minimize > minimization.yaml
# Edit the input paths and settings.
gmxtransplant prepare-minimization -i minimization.yaml
```

The folder contains coordinate and topology copies, all topology dependencies, an expanded topology for execution, a settings YAML, the Python runner and its support files, run.sh, generic multicore CPU and GPU SLURM templates, a README, and an input audit with checksums. Move the complete folder to your execution machine; GMXtransplant itself is not required there.

Settings

Every setting has a default suited to CHARMM36 membrane systems, so an assembly configuration needs only minimization: {enabled: true}. The settings you are most likely to change are:

```
minimization:
  enabled: true
  restraint_force_constant_kj_mol_nm2: 1000.0
  max_iterations: 5000
  platform: auto           # or CPU, CUDA, HIP, OpenCL
  restraint_residue_classes: {} # e.g. {MYLIP: lipid} for a custom lipid
```

Nonbonded, constraint and SLURM resources settings remain available in YAML for special cases; the GUI shows only the settings above.

Restraints use the input coordinates as references and a default force constant of 1000 kJ/mol/nm². Protein

molecules are recognized from their amino-acid residues, including attached caps and modifications. Unrecognized molecules are treated as ligands. Classify custom lipids and unusual residues explicitly, for example `restraint_residue_classes: {MYLIP: lipid}`. Supported classes are protein, ligand, lipid, water, and ion. The runner uses OpenMM L-BFGS with a finite iteration limit and an RMS objective-gradient tolerance. Review the example's nonbonded settings for your force field. OpenMM potential switching differs from LJ force switching; this is a configurable clash-relaxation step, not a universal simulation protocol. Topology position_restraints must be inactive because the runner adds its own restraints. Preparation rejects active definitions instead of ignoring them.

Run separately

From the generated folder:

```
cd openmm_minimization
python3 -m pip install -r requirements.txt
bash run.sh
```

The default run uses multiple CPU threads. Set `OPENMM_CPU_THREADS` to adjust the local thread count, or select a GPU with `bash run.sh --platform CUDA`. HIP and OpenCL can also be selected when available. For SLURM, review the resource directives and software setup, then submit one template from this folder:

```
sbatch submit_cpu.slurm
# Alternatively:
sbatch submit_gpu.slurm
```

The CPU template uses one process with multiple threads. The GPU template requests one GPU and selects CUDA. An unavailable requested platform produces an error.

Inspect results

The runner writes:

- `results/minimized.gro`
- `results/minimization_report.txt`
- `results/minimization_report.json`

Reports include convergence, energies, forces, selected restraint counts, inferred ligand names, and heavy-atom displacement. Energies include the positional restraints. Exit code 7 means failure or unconfirmed convergence; inspect the report before using any output coordinates.

Existing preparation and results directories are never overwritten. Choose a new destination for preparation, or preserve an earlier results/ directory by renaming it before rerunning. Minimization does not replace equilibration.

Assembly checks and outputs

Assembly checks the periodic environment, aligns the incoming component, handles clashes, optionally corrects charge, and rebuilds the topology molecule list.

Clash cutoffs act on complete environment residues. Every mode uses the same `clash_detection.threshold` of 0.80 Å by default, and no molecule class carries an implicit cutoff of its own: cholesterol is judged exactly like any other lipid. Override a single class through `clash_detection.thresholds` when a system needs it. Classes in `clash_detection.keep_classes` are retained and their contacts are reported. Neutralization removes eligible existing counterions; it does not add ions or set a new salt concentration.

Cholesterol is treated as a distinct molecule only in cholesterol-restoration mode, which is the mode that reasons about it. In protein, ligand and CHARMM-GUI transplant mode it is classified, coloured and reported as an ordinary membrane lipid.

A successful run produces GRO/PDB coordinates, an inspection PDB, text/JSON reports and index.ndx, plus topol.top and toppar/ when topology writing is on (as in the examples). Review the alignment, removed molecules, retained clashes, lipid counts, net charge, and coordinate/topology checks before continuing.

The commented example files describe box validation, name restoration for truncated PDB names, clash thresholds, ion protection, and output options.

Limitations

- All components must already have compatible parameters. GMXtransplant does not parameterize new molecules.
- Validated periodic processing supports orthorhombic boxes. Use GRO as the authoritative simulation coordinate output.
- Experimental bound-lipid restoration supports cholesterol only; other bound lipids have no dedicated preservation/restoration option.
- Composition correction removes surplus lipids but does not build missing lipids or solvent.
- Conditional topology definitions may need preprocessing. Minimization preparation supports literal includes, object macros, and `#ifdef/#ifndef` branches; unsupported expressions and cyclic includes are rejected.
- Preparation checks input consistency without importing OpenMM. OpenMM parsing and force-field compatibility are checked when the runner executes.
- Exact overlaps, poor placement, or incorrect parameters may need correction before minimization can succeed.

Development

```
python3 -m unittest discover -v
python3 -m pip install build
python3 -m build
```

GitHub Actions tests Python 3.10, 3.11, and 3.12, builds the package, and runs OpenMM tests with the optional dependency installed.

Runnable protein insertion example

The repository includes a complete input system in `examples/protein_insertion` (`examples/protein_insertion/README.md`). Its relative paths are resolved against the installed example folder, so it runs from anywhere:

```
python -m pip install .
gmxtransplant --mode protein \
-i "$(python3 -c 'from gmxtransplant.gui.model import example_path; print(example_path("protein"))')" \
--output ~/gmxtransplant-example
```

`environment/` supplies the target box, membrane, solvent and ions. `replacement/` supplies the selected incoming protein and bound ligands only. Protein topology definitions come from the `replacement`; retained environment molecules use environment definitions. Explicit incoming ligand ITPs supersede old copies in either directory. Conflicts within a source remain errors.

In the generated examples, incoming ligand names and ITP paths are together near the top. `charge: from_itp` and `moleculetype` need not be repeated. Coordinate `resname` and ITP `[ moleculetype ]` names need not be identical: the unique matching `[ atoms ]` residue name determines the type. Ambiguous ITPs still require an explicit molecule type. Old detailed YAMLS remain supported.

For protein-mode PDB name restoration, the environment's `topol.top` is found beside `environment_toppar_dir`; no separate reference topology is needed. A nonstandard layout can still set

name_restoration.reference_topol explicitly. The example enables name restoration so truncated POP/TIP/CHL names are classified correctly before clash removal. Scientific choices (masks, cutoffs, protection and composition) remain explicit in the YAMLS; inactive/default plumbing is omitted. Standalone minimization settings remain in minimization.yaml.

Desktop application

The desktop application is the main way to use GMXtransplant. It edits the same configurations and runs the same pipeline as the command-line tool, so anything set up here can also be run with `gmxtransplant` in scripts or on a cluster. It runs locally on macOS, Linux, or inside WSL 2 with WSLg. It does not upload input files or run a web server.

GMXtransplant input editor

Install and launch

From the source directory containing `pyproject.toml`:

```
python -m pip install .
gmxtransplant-gui
```

This one installation provides both the desktop application and the `gmxtransplant` command-line tool. For editable development, use `python -m pip install -e ..`. From a wheel, use `python -m pip install /path/to/gmxtransplant-1.0.1-py3-none-any.whl`. The equivalent module launch is `python -m gmxtransplant.gui`. Activate the same Python environment you used for installation before launching.

Qt (PySide6) is installed with the package but only loaded when the application is launched, so the command-line tool also runs on machines without a display. Where PySide6 cannot be installed, `GMXTRANSPLANT_CLI_ONLY=1 python -m pip install .` installs the command-line tool alone; add the application later with `python -m pip install "PySide6>=6.6,<7"`. The complete molecular example datasets ship with the package so installed examples also work offline. Example data adds approximately 58 MB on disk; the complete wheel is about 12 MB compressed.

On Linux, launch from a graphical desktop session. On Windows, install and launch inside WSL 2 with WSLg, using the Linux Python environment. Windows 10 build 19044+ or Windows 11 is required for Microsoft's integrated Linux GUI support. See Microsoft's WSL GUI setup (<https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps>). The chooser can browse Linux paths and mounted Windows drives such as `/mnt/c/` and `/mnt/f/`. Use Linux paths inside WSL, rather than `C:\...` or `F:\....`. A headless SSH session needs display forwarding; the CLI remains usable without a display. Linux distributions may require the system libraries used by Qt's X11/Wayland platform plugin in addition to the Python installation.

Your first run

1. Check the Output folder at the top. It starts as the folder you launched `gmxtransplant-gui` from, just as the command-line tool writes to the folder it runs in; choose another with Choose folder.... Checking or running without one shows a message at the top of the window explaining what is missing.
2. Choose a mode: CHARMM-GUI protein transplant (first), protein, ligand, cholesterol, or standalone minimization preparation.
3. Start from the mode's template, or open a complete example from the Examples tab with Open in editor. Placeholder inputs must be replaced with your own files.
4. Use the Input editor to choose every input. Each field holds one complete path: browse for it, or type it in full. There is no input base folder, and nothing is resolved against a shared root or the working directory, so an incomplete path is reported before the run starts, naming the field. Expand configuration sections to edit selections, alignment, topology, composition, and other settings.
5. Use Check configuration for schema/values, then Check input paths for the existing loader's input checks. Neither performs full assembly validation.

6. Click Run pipeline. The Run & results tab shows a Progress at a glance

box with the current stage, key results (such as alignment and charge), and scientific warnings or errors. Detailed output is collapsed on every new run; click View Command Progress to expand it and Hide Command Progress to collapse it. The log keeps collecting while hidden. When the run ends, the absolute folder the outputs were written to is printed above the file list and in the log; double-click any generated file to open it.

The form exposes current settings even when omitted by a template. Every setting has a one-line explanation under it saying what it does and what you can change. Deprecated settings are not shown. Each setting has exactly one control:

- Settings with a default are shown pre-filled. They are written to the YAML only when they were in the loaded file or you change them.
- Settings the pipeline detects or infers (such as protein molecule names or box dimensions) have one checkbox. Unticked, the pipeline decides; ticked, the field for your value appears.
- Sections the pipeline switches on and off (topology, residue name restoration, minimization) have one checkbox for the whole section. The index.ndx index file is always written, so it has no switch.
- On/off settings are a single checkbox; numbers, dropdowns for supported methods, and file/folder choosers are used elsewhere.
- Lists of names or numbers are one text field, for example PROA, PROB.

Named entries (residue tables, environment overrides) have an inline name field and Add button, which appear once the setting is ticked.

- Output folders are not shown: every output follows the output folder chosen at the top of the window.

Residue labels retain uppercase names such as CHL, POP, and TIP. Only settings accepting genuinely different values offer a choice, such as reading a ligand's charge from its ITP or entering a numeric charge.

The minimization section shows only the restraint strength, iteration limit, OpenMM platform, and custom residue classes; everything else uses defaults. Protein and ligand heavy atoms and lipid head groups are restrained, while lipid tails, water and ions relax.

Use Theme in the top-right corner to switch between the Teal, Midnight (dark) and Sand colour themes. The choice applies to every window, including file choosers, and is remembered.

Messages and errors appear in a banner inside the window rather than in pop-up dialogs, and file choosers never block the main window. Under WSLg a modal pop-up can open behind the main window, which previously made the application look frozen.

There is no YAML editing in the application: the form builds the configuration in the background. Each run saves it as run.yaml in the output folder, with every input as a complete path, so the same setup can be repeated from the command line with `gmxtsplant --mode -i run.yaml`. Existing YAML files are run with the command-line tool. Switching modes retains each mode's edits.

Readable progress with collapsed command output

Documentation

Documentation in the left panel opens the complete PDF documentation that ships with the package. It is resolved relative to the installed package, never to a location on your computer. The button tries your desktop's default PDF viewer first (the Windows default viewer under WSL), falls back to your default web browser, and reports a clear error if neither is available. The viewer opens in the background; the GUI never waits for it.

macOS and MacBook Air

Use a graphical macOS desktop session and a native Python 3.10+ installation matching your machine (arm64 on Apple Silicon, x86_64 on Intel). macOS 13+ is the recommended baseline for current Qt releases; check the Qt platform requirements (<https://doc.qt.io/qt-6/supported-platforms.html>) for the particular PySide6 version installed. Qt supplies native macOS wheels; XQuartz and Linux DISPLAY/WAYLAND_DISPLAY variables are not needed. See Qt for Python installation (<https://doc.qt.io/qtforpython-6.8/gettingstarted.html>).

From the repository root, these commands work in Bash and zsh:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install .
gmxtransplant-gui
# Equivalent launch:
python -m gmxtransplant.gui
```

For an existing installation, activate its environment and run `python -m pip install --upgrade .` from the repository root. Close and reopen the application after upgrading.

The cholesterol example needs Open Babel. On macOS 14 or newer it is installed together with GMXtransplant. On macOS 12 and 13 install it with Homebrew, `brew install open-babel`, then verify `obabel -V` in the same terminal used to launch the GUI. See the Homebrew Open Babel formula (<https://formulae.brew.sh/formula/open-babel>). The protein and ligand examples do not require Open Babel. PyMOL remains optional for its specific alignment methods; the examples use the NumPy-based mask fit.

A macOS GUI test job is included in CI alongside Linux. Local verification for this release used Linux with Qt's offscreen display, not a physical Mac.

Output layout and replacement

```
selected_output_folder/
+-- protein/
+-- ligand/
+-- cholesterol/
+-- minimization/
|   +-- openmm_minimization/
+-- examples/
    +-- protein/
    +-- ligand/
    +-- cholesterol/
```

Folders are created only when used. There are no timestamps or nested run folders. Coordinates, `topol.top`, `toppar/`, index files, reports, and enabled diagnostics go directly into the applicable mode directory. The GUI uses the configured output filenames, but redirects output directories to this layout. A portable minimization bundle requires its own `openmm_minimization/` directory.

Every run writes `run.yaml` (the exact configuration with resolved absolute input paths), `run.log`, and `run-status.json` beside the products. The snapshot can be run through the CLI. Source input files stay in their original locations. Shared `$(name)` references stay intact in the editor and are resolved in the run snapshot.

Repeating a run replaces previous generated products automatically. This includes the generated `toppar/` and minimization bundle, so stale ITP files do not survive. No overwrite confirmation is required. Output paths overlapping source inputs and symlinked generated destinations are rejected. Unrelated files in the output folder are retained. A hidden manifest tracks earlier generated filenames; a lock prevents two GUI runs from writing to the same mode folder simultaneously.

Configuration/input validation runs before replacement, preserving previous outputs when these checks fail. Once a full run starts, previous products are removed and a failed/cancelled run may leave partial new products. Inspect the run status and log; file existence alone does not indicate success. Avoid editing inputs or

running another application against the same outputs during a run.

Examples

Offline example cards

The Examples tab contains CHARMM-GUI transplant, protein, ligand, and cholesterol cards. After selecting an output folder, Run example uses the complete supplied inputs in one click and writes to examples/charmprot/, examples/protein/, examples/ligand/, or examples/cholesterol/. Reruns replace the same products. The GUI's mode editor is not changed by running an example.

Open in editor loads the example for customization. A subsequent editor run uses the normal mode directory (for example cholesterol/). Example inputs in the installation remain unchanged.

The cholesterol example requires Open Babel, which is installed with the package on Linux and macOS 14+ (see the README otherwise). Other optional scientific dependencies are exactly those required by the CLI settings: PyMOL for its alignment methods and OpenMM for running an exported minimization. Minimization preparation itself does not launch a simulation.

Cancellation and testing

Cancel run interrupts the worker and its external tools. A worker that does not stop is killed after five seconds. Closing the window while running also cancels the job. The GUI remains responsive because scientific work executes in a separate process using the same Python environment.

Developers can run the focused tests without a display:

```
QT_QPA_PLATFORM=offscreen python -B -m unittest -v test_gui
```

Qt-dependent tests skip automatically if PySide6 cannot be imported. Model/path tests still run.

Two-folder transplant and comparison views

The first mode requires only Reference and Transplant folders. Each must contain topol.top, toppar/, and step5_input.gro (or a unique step5*.gro). Optional selections and numerical settings are under Advanced settings. The default replaces all reference proteins and ligands with their transplant counterparts. Outputs go directly to charmprot/, or examples/charmprot/ for its supplied example.

Output folders are not shown in the form: every output follows the output folder at the top of the window. The input-file list has no Add or Remove buttons; it edits the named paths the configuration actually uses. Named entries such as residue mappings keep an inline Add control.

Open in PyMOL and Open in VMD are shown only when that viewer is installed, and become available after a successful run. A viewer is found on the PATH, in the Python environment GMXtransplant runs in, or on macOS as an application in /Applications or ~/Applications (for example PyMOL.app or VMD 1.9.4....app). To use a copy installed elsewhere, set GMXTRANSPLANT_PYMOL or GMXTRANSPLANT_VMD to its full path before launching. Each opens the generated comparison scene. The coordinate pipeline does not require either viewer. See the visualization guide (VISUALIZATION.md) for the color legend and aligned input overlays.

CHARMM-GUI transplant mode

This is the first CLI/GUI workflow. Run the installed package:

```
gmxtransplant --mode charmprot -i charmprot.yaml --dry-run
gmxtransplant --mode charmprot -i charmprot.yaml
```

Or from the source checkout:

```
python -B run_pipeline.py --mode charmprot -i charmprot.yaml --dry-run
python -B run_pipeline.py --mode charmprot -i charmprot.yaml
```

The entire required YAML is:

```
charmprot:
  reference: /path/to/reference/gromacs
  transplant: /path/to/donor/gromacs
```

Each folder must contain `topol.top`, `toppar/`, and `step5_input.gro` (or exactly one `step5*.gro`). Both folder paths, and any `reference_gro/transplant_gro` override, must be written out in full: nothing is resolved against the YAML's directory or the working directory. Inputs are read-only. `output_dir` defaults to the name `charmprot_output`, placed in the `--output` directory when you pass one and otherwise in the directory you ran the command from. Rewriting an existing `output_dir` is the norm: generated artifacts are replaced transactionally, with rollback if publication fails. Unrelated files and GUI logs/locks are preserved. A failed assembly publishes failure reports and removes stale generated results. `--dry-run` validates the YAML without opening the input folders.

Detection and selection

The program reads the actual topology include graph and expands [molecules] using ITP atom counts. Every coordinate atom must agree with its ITP residue and atom name in file order. GRO serial-number wrapping does not affect selection. Unreferenced ITP files do not affect classification.

PROA, PROB, and other PRO... topology molecule types identify proteins. An additional backbone check recognizes protein types with other names. Every detected protein is selected by default. A covalent cofactor inside the same protein moleculetype travels with that molecule; ligands: ignore only applies to separate ligand molecules of the transplant.

Environment categories are water, ion, lipid, sterol, detergent and solvent. The bundled catalogue is a dated snapshot of 943 distinct identifiers from 19 primary-source pages, supplemented with exported water/ion/lipid aliases. It includes every download identifier on the retrieved Individual Lipid Library page and every RESI identifier in the retrieved lipid, water/ion and noble-gas topology files linked from the CHARMM-GUI topology index. It does not treat patch identifiers (PRES) as complete molecules.

Primary sources:

- CHARMM-GUI Individual Lipid Library (<https://www.charmm-gui.org/?doc=archive&lib=lipid>)
- CHARMM-GUI topology index (<https://www.charmm-gui.org/?doc=toppar>)
- Water and ions topology
(https://www.charmm-gui.org/?doc=open_toppar&filename=toppar_water_ions.str)
- Extended ion topology
(https://www.charmm-gui.org/?doc=open_toppar&filename=stream/misc/toppar_ions_won.str)
- CHARMM-GUI water-model FAQ (<https://www.charmm-gui.org/?doc=faq>)
- OpenMM CHARMM water distribution
(<https://github.com/openmm/openmmforcefields/blob/main/charmm/files/waters.yaml>)

The catalogue records a retrieval date, source hashes and per-name source references in `gmxtransplant/charmm_environment.json`. `charmm_environment.py` contains export aliases. Ordinary runs

work entirely offline.

Names are matched exactly, case-insensitively. Both molecule names and the ITP residue names are examined. Thus all catalogued POP... lipids are covered without an unrestricted POP* rule. Single charged atoms also qualify as ions even when their name is custom. Unknown non-protein molecules become inferred ligands and are named explicitly in the report. Topology position after the protein and before the environment is recorded as supporting evidence, not proof of chemical identity.

No fixed name list can guarantee every future, custom or renamed CHARMM-GUI component. The snapshot's coverage is stated precisely above; unfamiliar names remain visible in the report. Use an explicit ligand list or environment override when a name has a different biological role (e.g. a bound metal or functional cholesterol molecule).

Optional YAML

```
charmprot:
  reference: /path/to/reference/gromacs
  transplant: /path/to/donor/gromacs
  reference_proteins: [PROA, PROB]
  transplant_proteins: [PROB, PROA]
  reference_ligands: [LI1]
  transplant_ligands: [LI2]
  output_dir: transplant_result
```

Protein lists specify which complete molecule types to replace and the chain pairing order. Multiple instances of a selected type are all selected, in file order. Omitted protein types in the reference remain in place. Ligand lists take precedence over the environment catalogue; [] explicitly selects no ligands on that side. Omission means automatic detection on that side.

ligands chooses what is inserted from the transplant:

- auto (default): the reference's proteins and ligands are removed, and the

transplant's proteins and ligands are inserted.

- ignore: the reference's proteins and ligands are removed in the same way,

but only the transplant's proteins are inserted; its ligands are left out. reference_ligands may still name which reference ligands to remove; transplant_ligands cannot be combined with ignore.

If a retained protein or ligand (one not selected for removal) clashes with the incoming block, the run stops and reports the conflict.

Additional options:

Option	Default	Meaning
reference_gro, transplant_gro	automatic	Full path to a coordinate file, when the folder's own step5*.gro is not the one to use
environment_overrides	{}	Explicit molecule-name to category mapping, e.g. {MYLIP: lipid}
clash_distance	0.80	Heavy-atom environment clash distance in Å; removes complete molecules
keep_lipids	false	Keep every lipid and sterol, even when it clashes; only other clashing molecules (water, ions, solvent) are removed. Retain
neutralize	true	Remove eligible counterions to restore target charge
target_net_charge	0.0	Desired charge in elementary-charge units
ion_exclusion_distance	10.0	Minimum ion distance from inserted/retained biological molecules in Å
lipid_ion_exclusion_distance	5.0	Minimum ion distance from retained membrane heavy atoms in Å

Alignment, topology and outputs

Corresponding protein chains are globally aligned by residue sequence, followed by a single C α Kabsch rigid fit over all matched chains. This supports missing residues and substitutions without PyMOL. Chain counts must agree; low-identity pairings stop with a clear error. Symmetric chain permutations are not searched automatically: give corresponding protein lists if file order differs. All selected donor atoms, including ligands, receive exactly the same transform. The donor protein should be whole under periodic boundaries before alignment.

Environment clashes use minimum-image distances with the reference box. Whole topology molecules are

removed, including multi-residue lipids. A 0.02 Å margin allows for GRO rounding. Exact molecule charges come from ITP [atoms] records. Charge balancing removes distant counterions with the needed charge; it stops if the requested charge cannot be reached. It does not add ions.

The parameter sets are combined, identical definitions are deduplicated, and conflicting definitions stop the run. Protein and ligand molecule definitions come from the donor; retained molecules keep reference definitions. Conditional include graphs, parameter macros and nested molecule includes need preprocessing; unsupported ambiguity is rejected. Standard flat CHARMM-GUI toppar inputs are the intended format.

On success the output folder contains:

- step5_input.gro, step5_input.pdb: combined coordinates in the reference box
- topol.top, toppar/: assembled parameters and molecule definitions
- index.ndx: topology-based protein, ligand, environment and component groups
- aligned_inspection.pdb: reference and transformed donor selection overlay
- charmprot_report.txt, charmprot_report.json: both systems' classifications,

selection decisions, unknown names, alignment, molecule removals and charge audit

The final topology is reparsed independently and checked against the written GRO atom order and charge. Final files are published together from temporary storage. On failure only diagnostic reports are published, with status: failed. The reports are the record of exactly what was considered environment and ligand.

--prepare-minimization also exports a portable minimization bundle after a successful transplant; it does not run minimization.

GUI, examples and visualization

The first mode presents Reference and Transplant folder pickers. All other settings are under Advanced settings; the output folder is selected once at the top of the window. The supplied datasets are in examples/charmprot/. Its Examples card runs them in one click.

Canonical configurations use a charmprot: section and may use the shared paths: registry. The original flat two-folder YAML is still accepted. A root minimization: section uses the same settings as other modes; --prepare-minimization is also supported.

All successful modes produce view.pml, view.vmd, and a visualization/ asset directory. A view.pse session is also created when PyMOL is installed. See Visualization (VISUALIZATION.md) for colors and input overlays.

Comparison views

Each successful assembly exports view.pml, view.vmd, and visualization/ alongside its coordinates. A real view.pse session is generated automatically when PyMOL is installed; PyMOL is found on the PATH, in the running Python environment, or on macOS in /Applications, and GMXTRANSPLANT_PYMOL can point to any other copy. PyMOL and VMD are optional external viewers; neither becomes a CLI or GUI dependency. Minimization preparation shows the prepared input, not an unperformed minimization result.

Use Open in PyMOL or Open in VMD in GUI results, or run from the output folder:

```
pymol view.pml
vmd -e view.vmd
```

Every mode that produces a visualization writes view.vmd and view.pse into its output folder, alongside view.pml and the visualization/ assets, so the assembled system can be reopened later without rerunning the pipeline. view.pse is written during the run when PyMOL is on PATH; when it is not, the run says so and opening view.pml saves view.pse. The PSE is self-contained. When moving a scripted scene, copy the entire output folder, including visualization/, and launch from that folder. VMD locates its assets relative to the .vmd file. For Python-script loading in PyMOL, use run visualization/load.py, module.

Component	Color	Initially visible
Incoming protein	Blue	Yes
Retained protein	Slate	Yes
Incoming ligand	Magenta	Yes
Retained ligand (topology-classified transplant mode)	Purple	Yes
Incoming cholesterol	Gold	Yes
Retained cholesterol	Teal	Yes
Retained membrane	Pale gray	Yes
Original replaced protein	Orange	No
Original replaced ligand	Orange	Yes
Removed environment	Red	No
Reference input	Amber	No
Aligned transplant input	Cyan	No
Water / ions	Pale blue	No

Toggle named objects in PyMOL or molecules in VMD. The old ligand is a comparison overlay, not part of the final molecular system. Full input overlays and removed environment are initially hidden to avoid obscuring the result. visualization/LEGEND.txt and scene.json list the roles and atom counts actually present in that run.

Source overlays use the same rigid transformation as assembly, in the final protein's coordinate frame. Ligand mode shows the positioned incoming ligand; cholesterol mode shows the converted, aligned experimental structure. Box repair, when enabled, is already reflected in the reference overlay. These views do not apply a second fit, wrap molecules, or change scientific coordinates.

Inserted membership comes from the actual inserted atom block, rather than guessing by residue name: an old and a new ligand with the same residue name remain distinct. Two-folder mode additionally uses exact topology molecule categories. Other modes use their standard residue classification for display; unknown retained components are labeled "Other retained molecules." Viewer files are for comparison, not simulation inputs; use the validated final GRO and topology for simulation.

Viewer export failures are reported without invalidating successfully written scientific outputs. Details remain in View Command Progress. The generated scripts work without network access on supported viewer installations on Linux, WSL, and macOS.

Portable and offline use

Install from the source directory:

```
python3 -m pip install .
```

Create and edit a configuration in your job directory:

```
gmxttransplant --show-example protein > protein_replace.yaml
gmxttransplant --mode protein -i protein_replace.yaml --dry-run
gmxttransplant --mode protein -i protein_replace.yaml --prepare-minimization
```

Use modes `lig` or `chl` for ligand replacement or cholesterol restoration. Omit the preparation flag and leave `minimization.enabled: false` for assembly alone.

To prepare an existing matching system:

```
gmxttransplant prepare-minimization --coordinates step5_input.gro --topology topol.top
```

Both workflows create one `openmm_minimization/` folder. Preparation copies and audits the inputs; it does not minimize or submit a job. The folder includes all topology dependencies, the standalone runner, settings, and generic CPU/GPU SLURM templates.

Move the whole folder to your execution machine, activate your Python environment, and review the settings:

```
cd openmm_minimization
python3 -m pip install -r requirements.txt
bash run.sh
```

The runner relaxes clashes with harmonic restraints on protein and ligand heavy atoms and lipid head groups (polar P, O, N and S atoms). Lipid tails, water, ions, and hydrogens relax freely. Classify custom lipids with `minimization.restraint_residue_classes` so they are not treated as ligands.

For SLURM, edit and submit either `submit_cpu.slurm` or `submit_gpu.slurm` from this folder. Inspect `results/minimized.gro` and the convergence reports before continuing. Existing inputs and results are never overwritten.

See `README.md` (`README.md`) for all modes, settings, and limitations.